

ProofOps

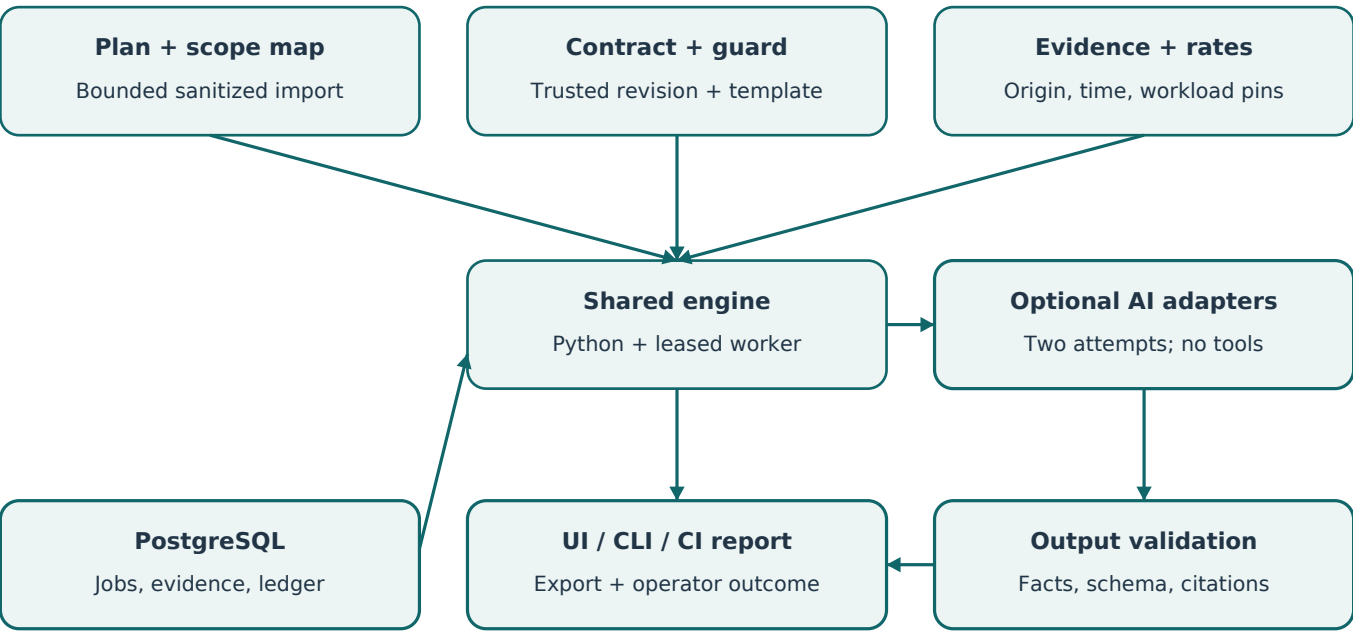
Project overview

Evidence for a specific infrastructure change

Working local software. Synthetic review fixtures. Measured local Docker experiments. Live AWS and paid model evaluation unrun.

Build date: 2026-10-05 · Application version 0.1.0

Editable Markdown, machine inventories and saved result extracts accompany this searchable guide. Source links and section links are clickable.



Human review > draft fixtures > separate protected source review. No app deployment.

[Read the overview](#) · [Inspect recorded results](#)

ProofOps project overview

Implementation date: 5 October 2026. Status: working local application with tested replay, mocked cloud/model contracts and measured local workload experiments. Live AWS, paid model evaluation and uncoached human usability remain unrun. All shipped AWS review examples use visibly synthetic evidence, rates and approvals.

The problem and intended user

An engineer operating an ECS Fargate service wants to reduce task CPU or memory. A lower projected compute price alone does not show that the service still meets its operating requirements. The engineer also needs an exact Terraform change, service identity, recent demand/latency evidence, workload tests and any applicable incident lesson.

ProofOps turns those separate artifacts into a review with explicit findings, coverage and a next step. Its question is: can this specific change reduce the covered task charges while meeting this service's stated contract, and what evidence supports that answer? It starts from a plan or replay bundle, not an open-ended chat. An engineer remains responsible for review and deployment.

From a plan to an inspectable decision

Before using the app, the engineer compares plan JSON, cost assumptions, telemetry, performance runs and past incidents manually. With ProofOps, they import a bounded bundle, inspect one report, reproduce its deterministic result and export a constrained guard draft for separate review. Recorded outcomes make the later operator decision visible alongside its evidence.

The engine has four outcomes: revise a known violation, collect missing/incompatible evidence, request engineering review when supported checks pass, or report unsupported scope. Errors and malformed imports are execution failures. AI is optional: it can explain facts or format an approved guard proposal, but cannot invent a safe memory bound, change a price or activate a rule.

An illustrative review

The supplied valid replay moves from 2 vCPU / 4 GiB to 1 vCPU / 2 GiB. With unchanged task-hours and synthetic rates, task CPU/memory charges are estimated to fall by 50%. The rates are illustrative, not current AWS quotes. The synthetic candidate also satisfies its approved-in-fixture 2,048 MiB floor and complete fixture workload evidence, so the result requests engineering review.

A second synthetic candidate uses 0.5 vCPU / 1 GiB. Its arithmetic suggests a larger reduction, but the same scoped floor rejects it. A third replay supplies insufficient evidence; a 20-request smoke check cannot satisfy a 10,000-correct-request contract. Neither a good-looking saving nor a fluent model explanation overrides those results.

The estimate excludes network, logs, load balancers, storage, taxes and commitments. Different task-hours or incompatible test windows prevent a simple proportional/unit-cost claim. A projected task-charge difference is not an invoice or realized saving.

Implemented architecture and AWS connection

A React/TypeScript web UI connects to a Python FastAPI API. PostgreSQL stores typed/versioned records, jobs, reports, outcomes and budget accounting. A persisted worker runs the shared deterministic engine. The same engine powers the CLI and a CI summary. Fixed scoped Rego/Conftest checks provide one reviewed memory-floor policy family. Sanitized bundles reproduce history without a fresh model call.

The optional boto3 collector verifies the account through STS, then reads one mapped ECS service, CloudWatch metrics and bounded Logs Insights results. It has no deployment operations. OpenAI Responses and Anthropic Messages adapters are implemented behind a shared schema and two-attempt budgeted router. The default is AI off with empty keys and no exact live model ID. Missing live configuration does not block local replay.

The architecture diagram in this guide labels the deterministic path, optional AI path and separate human/policy review boundary. No research evaluator labels are mounted into the API/worker image or available to model context.

What the evidence establishes

Automated tests cover unknown/sensitive plans, scope, strict thresholds, credits/nulls, archive security, policy fixtures, PostgreSQL races, worker crashes, provider failures, model-output rejection and real browser workflows. A 60-case synthetic evaluation checks 20 groups split 8 development / 4 calibration / 8 held-out. The template and deterministic results are recorded; paid provider policies remain explicitly unrun. A schema/citation pass is separate from semantic correctness.

The local k6 experiment performs smoke, calibration and three baseline/candidate pairs with frozen inputs. A separate bounded pressure test records actual Docker OOM and repair. These are Linux ARM64 local observations; they do not establish an AWS Linux x86_64 operating bound. This guide's measured table is generated from saved artifacts and never represents billed cloud savings.

Scope, alternatives and limits

The implemented scope is one mapped Linux on-demand Fargate service, account and region, one task CPU/memory cost model and one memory-floor guard family. Multi-cloud discovery, Kubernetes, autonomous production changes/rollback, general diagnosis, arbitrary generated policy, public SaaS, multi-tenant identity and a trained router are deferred.

Infracost, AWS DevOps Agent, Compute Optimizer and existing incident/CI tooling already cover related work. ProofOps demonstrates a reproducible link between a proposed change, evidence, an approved constraint and an operator outcome. It makes no worldwide novelty or commercial-superiority claim. Protected source-control/workflow review, authentication, secure secrets and retention would be additional requirements for team hosting.

Data and ownership

Three replay bundles are original synthetic fixtures. The public FOCUS sample is attributed under CC BY 4.0 and tests exact Decimal billing ingestion; it is unrelated to the demo service's actual costs. Curated development cards/examples come from the supplied original ProofOps research. RCAEval labels and fault-bearing paths remain outside runtime. Research files were preserved with a checksum audit.

Start with the README's Docker instructions, then perform the owner walkthrough: valid saving example, rejected reduction, missing evidence, guard validation/export, changed-commit applicability and outcomes. Show measured local experiments and unrun model cells honestly. The technical guide explains every module, actual tools, model slots, tests, learning stages and operations needed to maintain the implementation.

Sources and further reading

[AWS Fargate pricing](#) · [Task sizing](#) · [Terraform JSON](#) · [FOCUS sample](#) · [SRE canaries](#) · [Infracost](#) · [Compute Optimizer](#) · [OpenAI structured outputs](#) · [Anthropic structured outputs](#).

Recorded results

Observed local verification, recorded 2026-10-05T01:42:22.103526+00:00. The technical guide and docs/results JSON contain complete commands, denominators and per-class measurements.

Check	Observed result
Backend / policy / PostgreSQL	94/94 passed
Real Chromium workflows	6 passed
Fresh checkout and frozen runtime install	Passed; frozen runtime install and keyless review/replay
Deterministic synthetic evaluation	60/60 cases, 20 groups, split 8/4/8
Terraform local validation	0 errors, 0 warnings; not deployed
Original research preservation	384 files unchanged

Measured local workload

Three alternating eight-minute pairs used one pinned Linux ARM64 Docker image/profile/dependency state on a shared development host. Baseline: 2 CPUs / 512 MiB. Candidate: 1 CPU / 256 MiB. Thresholds were frozen before candidates. This is not AWS x86_64 operating-bound evidence or measured cloud savings.

Run	Correct responses	p95 (ms)	HTTP failures
baseline-full-01	20,399	5.980	1
candidate-full-01	20,399	5.481	1
baseline-full-02	20,400	5.934	0
candidate-full-02	20,399	6.166	0
baseline-full-03	20,399	6.262	1
candidate-full-03	20,399	6.190	1

All six comparison runs passed: True. Required: p95 below 250 ms and HTTP failures below 1%. Observed incorrect successes / dropped work / restarts: 0 / 0 / 0. Separate bounded OOM plus repair confirmed: True; 128 MiB pressure at a 64 MiB limit, then 20 correct responses at 256 MiB.

Model evaluation status

The template run produced 57 structured explanations plus 3 coverage abstentions, and 54 applicable guard drafts plus 6 correctly inapplicable cases. These are narrow synthetic/template results. 360 paid-provider policy tasks are unrun; unrun quality/cost/latency stay null. Confidence or strong-model agreement never supplies truth.